MI SSL API

Version 1.2

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/12/2020
1.1	● Modified API	10/15/2020
1.2	● 4MIC Sound Source Localization and Modified API	11/03/2021

TABLE OF CONTENTS

1. Overview

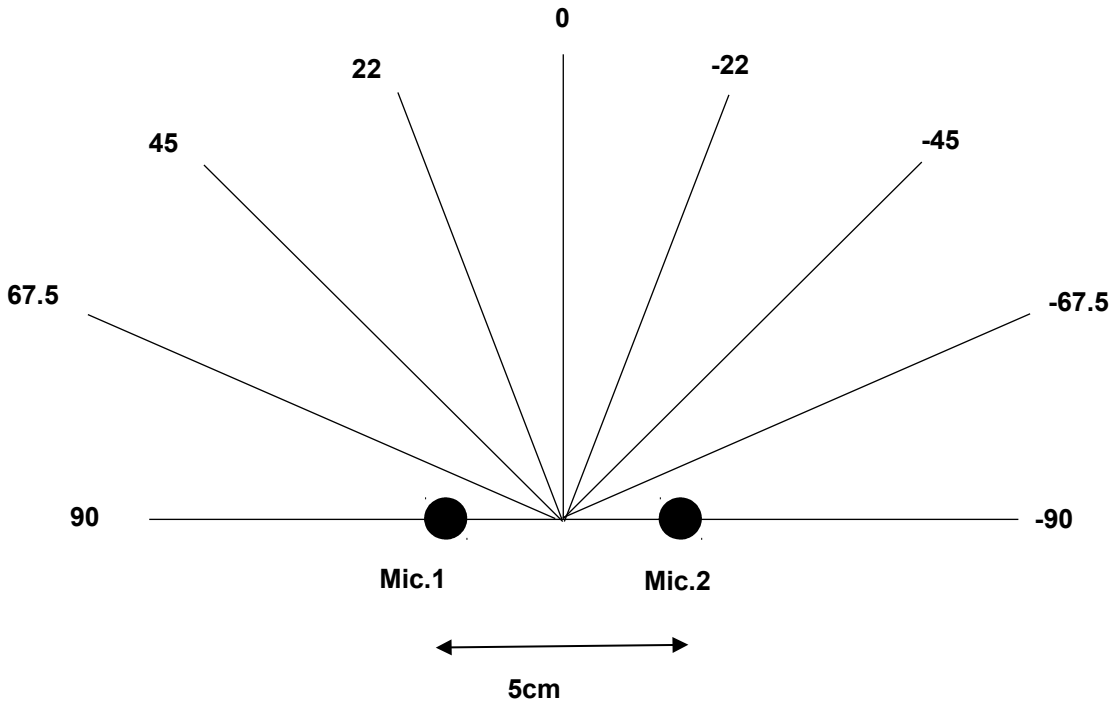
1.1. Algorithm Description

SSL (short for Sound Source Localization) is used to locate the direction of the sound.

1.2. Keyword

- Direction

When the distance between the two microphones is 5cm, the following figure describes the definition of the sound direction:



When the number of microphone is more than two, there are two possible configurations for the corresponding microphone array system: Uniform Linear Array (ULA) or Uniform Circular Array (UCA).

1.3. Note

1.3.1. Implementation

In order to facilitate debugging and confirm the effect of the algorithm, the user application needs to implement replacement algorithm parameter and grab audio data.

1.3.2. Correspondence

Different channel numbers will correspond to different reference libraries. The user needs to confirm whether to

use the correct number of channels and the corresponding reference library.

2. Coordinate system of Microphone Array

2.1. Multichannel Microphone array

There are two main types of multichannel array system used in practice, one is the uniform linear array, and the other is uniform circular array.

2.1.1. Uniform Linear Array

Uniform linear array is a straight line array that is spaced evenly. Because of its DOA symmetry, we only consider the sound direction locates at the upper plane (from -90 degree to 90 degree). Figure 2 illustrates the uniform linear array and its coordinate system. The sound direction is defined as the angle between the array center and the x-axis where counterclockwise is positive. We recommend the distance of adjacent microphones should be bigger than 5cm or 6cm.

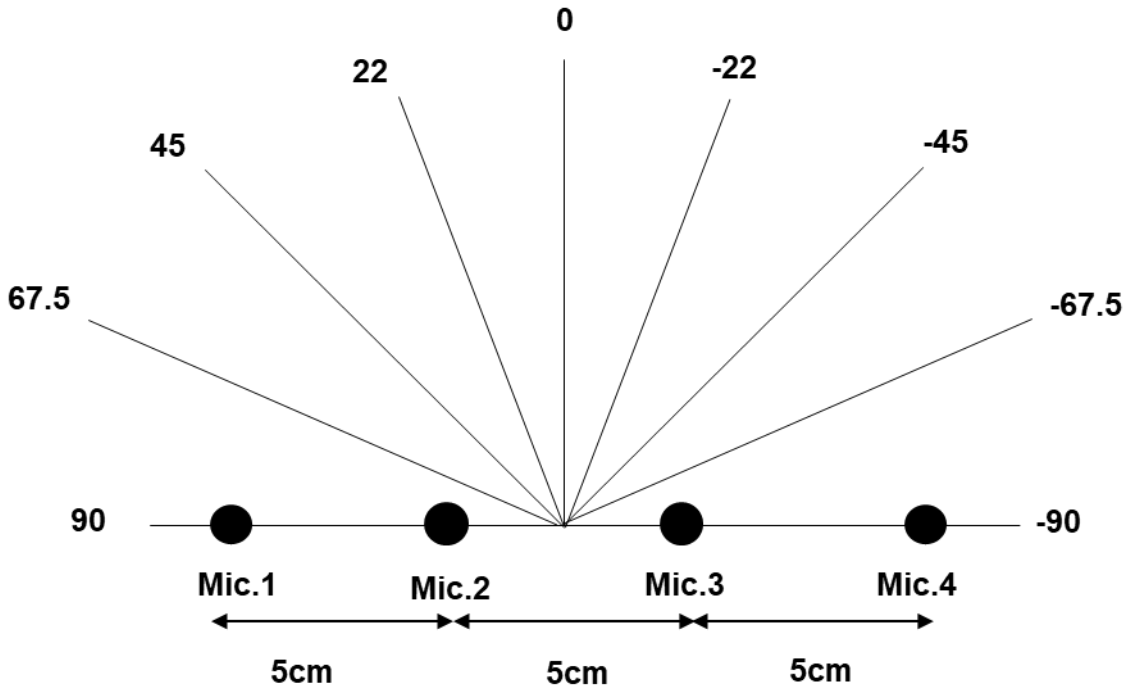
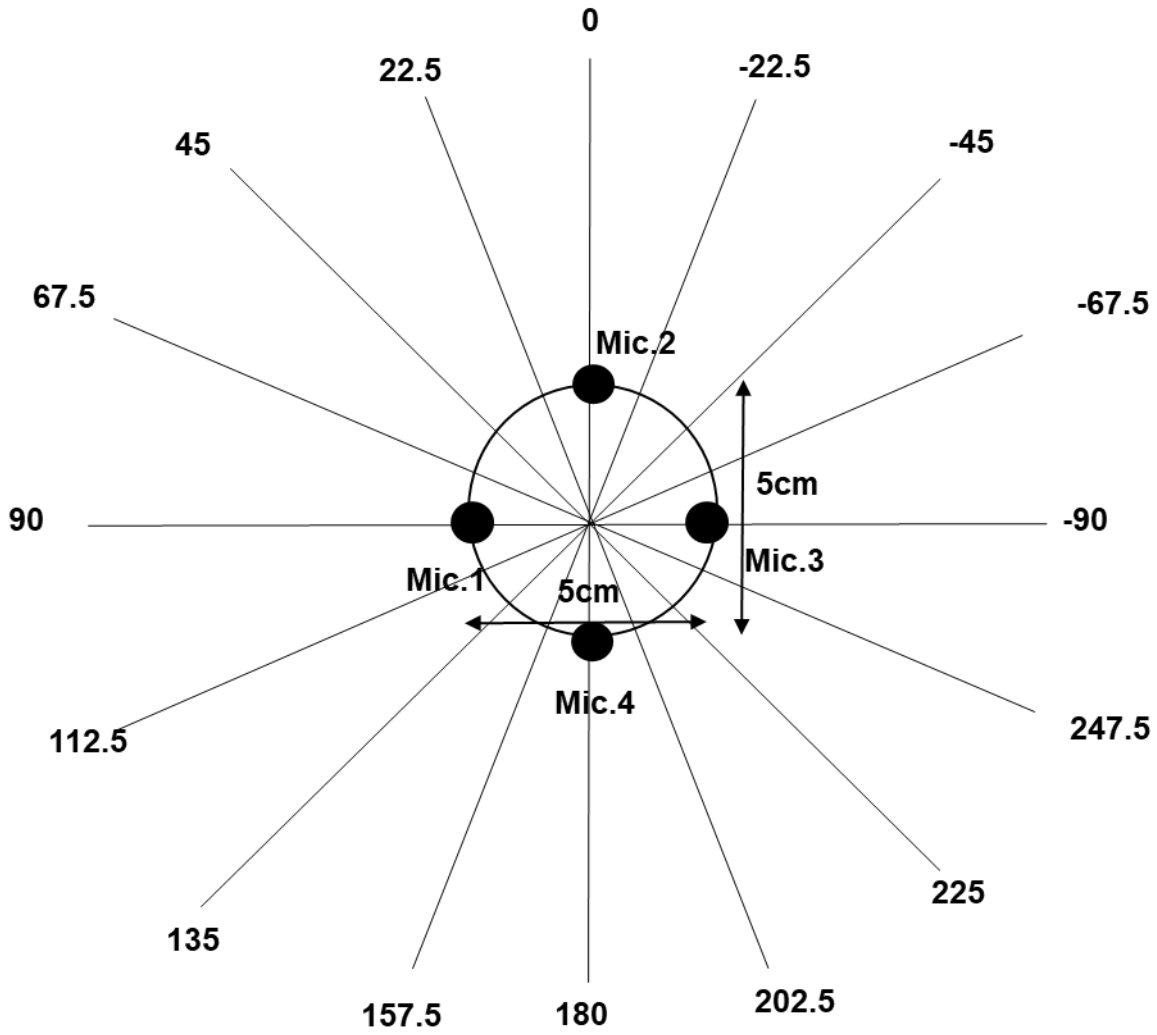


Figure 2. Uniform linear array and its coordinate system.

2.1.2. Uniform Circular Array

Uniform circular array is a circular array that the angle between adjacent microphones and array center are the



same. Because of its DOA asymmetry, we consider the sound direction locates at the whole plane (from -90 degree to 270 degree). Figure 3 illustrates the uniform circular array and its coordinate system. The sound direction is defined as the angle between the array center and the x-axis where counterclockwise is positive. The distance is the diameter of the circle. We recommend the distance should be bigger than 6cm.

Figure 3. Uniform circular array and its coordinate system.

3. API Reference

3.1. API List

API name	Features
IaaSsl_GetBufferSize	Get the memory size required for Ssl algorithm running
IaaSsl_Init	Initialize Ssl algorithm
IaaSsl_Config	Configure Ssl algorithm
IaaSsl_Get_Config	Get the current configuration parameter information of the Ssl algorithm
IaaSsl_Set_Shape	Define the type of array system belongs to either ULA or UCA.
IaaSsl_Cal_Params	Recalculate some parameters required for SSL algorithm according to the type of array system.
IaaSsl_Run	Ssl algorithm processing
IaaSsl_Get_Direction	Get the result direction of Ssl algorithm processing
IaaSsl_Reset_Mapping	Reinitialize the buffer after IaaSsl_Get_Direction.
IaaSsl_Reset	Reinitialize Ssl algorithm
IaaSsl_Free	Release Ssl algorithm resources

3.2. IaaSsl_GetBufferSize

- Features

  Get the memory size required for Ssl algorithm running.

- Syntax

  unsigned int IaaSsl_GetBufferSize(void);

- Parameters

Parameter Name	Description	Input/Output

- Return value

  Return value is the memory size required for Ssl algorithm running. The memory size is dependent on the microphone number.

- Dependency
  - Header: AudioSslProcess.h
  - Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

The interface only returns the required memory size, and the application and release of memory need to be processed by the application.

- Example

Please refer to IaaSsl_Run example.

3.3. IaaSsl_Init

- Features

Initialize Ssl algorithm.

- Syntax

```
SSL_HANDLE IaaSsl_Init(char* working_buffer,AudioSslInit* ssl_init);
```

- Parameters

Parameter Name	Description	Input/Output
working_buffer	Memory address used by Ssl algorithm.The memory address will be obtained after user applies for the memory size.	Input
ssl_init	Ssl algorithm initialization structure pointer	Input

- Return value

Return value	Result
Not NULL	Successful
NULL	Failed

- Dependency
  - Header: AudioSslProcess.h
  - Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

- Example

Please refer to IaaSsl_Run example.

3.4. IaaSsl_Config

➢ Features

Configure Ssl algorithm.

➢ Syntax

ALGO_SSL_RET IaaSsl_Config(SSL_HANDLE handle,AudioSslConfig* ssl_config);

➢ Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input
ssl_config	Ssl algorithm configuration parameter structure pointer	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

➢ Example

Please refer to IaaSsl_Run example.

3.5. IaaSsl_Get_Config

➢ Features

Get the current configuration parameter information of the Ssl algorithm.

- Syntax

  ALGO_SSL_RET IaaSsl_Get_Config(SSL_HANDLE handle,AudioSslConfig *ssl_config);

- Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input
ssl_config	Ssl algorithm configuration parameter structure pointer	Output

- Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

- Dependency
  - Header: AudioSslProcess.h
  - Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

- Example

Please refer to IaaSsl_Run example.

3.6. IaaSsl_Set_Shape

- Features

Define the type of array system belongs to either ULA or UCA.

- Syntax

  ALGO_SSL_RET IaaSsl_Set_Shape(SSL_HANDLE handle,int shape);

- Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input

shape	The integer to decide array shape of Microphone array 0: Uniform Linear Array. 1: Uniform Circular Array.	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

- Our lib only supports uniform linear array and uniform circular array (Figure 2 and Figure 3). User has to inform the modification if he/she requires special array geometry.
- Only uniform linear array exists when there are only two microphones.
- The settings of array position will impact largely on SSL performance. Therefore, the settings of array position must be matched to the microphone array being used.

➢ Example

Please refer to IaaSsl_Run example.

3.7. IaaSsl_Cal_Params

➢ Features

Recalculate some parameters required for SSL algorithm according to the type of array system.

➢ Syntax

ALGO_SSL_RET IaaSsl_Cal_Params(SSL_HANDLE handle);

➢ Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

- Dependency
  - Header: AudioSslProcess.h
  - Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

- Example

Please refer to IaaSsl_Run example.

3.8. IaaSsl_Run

- Features

Ssl algorithm processing.

- Syntax

```
ALGO_SSL_RET IaaSsl_Run(SSL_HANDLE handle,short* microphone_input, int *delay_sample);
```

- Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input
microphone_input	The microphone raw data	Input
delay_sample	The number of delayed samples for each microphone pair. It's recommended to be used when bf_mode is enable.	Output

- Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

-
- For dual microphone array (the microphone input is binaural data), the data pointed to microphone_input should use the sampling point as the smallest unit and be placed in the format of L,R,L,R … .The length must correspond to the point_number (the number of sampling points once Ssl process) set in IaaSsl_Init.
- For multichannel microphone array (microphone number is bigger than two where binaural data is not enough for processing), the input data of each microphone must be mono-channel. The data pointed to microphone_input should use the sampling point as the smallest unit and be placed in the format of [Left ➔ Right], according to the relative position. The length must correspond to the point_number (the number of sampling points once Ssl process) set in IaaSsl_Init.
- Take microphone array in both Figure 2 and Figure 3 as example, the microphone_input should be placed in the format of [MIC1➔MIC2➔MIC3➔MIC4].

- ExampleExample I: Dual Microphone array(The microphone input is a binaural audio)

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/time.h>
#include <sys/ioctl.h>
#include <stdlib.h>
#include "AudioSslProcess.h"

#define MIC_NUM (2)
#define USE_MALLOC   (1)
typedef unsigned char           uint8;
typedef unsigned short          uint16;
typedef unsigned long           uint32;


unsigned int _OsCounterGetMs(void)
{
    struct  timeval t1;
    gettimeofday(&t1,NULL);
    unsigned int T = ( (1000000 * t1.tv_sec)+ t1.tv_usec ) / 1000;
    return T;
}

int main(int argc, char *argv[])
{
    /*******Input file init*********/
    short input[256];
```

```
29.     char infileName[512];
30.     char outfileName[512];
31.     FILE * fin;
32.     FILE * fout;
33.     ALGO_SSL_RET ret;
34.     int counter2 = 0;
35.     unsigned int T0,T1,T2,T3;
36.     float avg = 0.0;
37.     float avg2 = 0.0;
38.     /**********common setting SSL***************/
39.     int point_number = 128;
40.     float microphone_distance = 12.0;
41.     int temperature = 20;
42.     int sample_rate = 16000;
43.     int delay_sample[1] = {0};
44.     int shape = 0;
45.     int direction = 0;
46.     int frame_number = 32;
47.     /**************SSL data init***********/
48.     int counter = 0;
49. #if USE_MALLOC
50.     char *WorkingBuffer2;
51.     WorkingBuffer2 = (char*)malloc(IaaSsl_GetBufferSize());
52. #endif
53.     AudioSslInit ssl_init;
54.     AudioSslConfig ssl_config;
55.     SSL_HANDLE handle;
56.
57.     ssl_init.mic_distance = microphone_distance; //cm
58.     ssl_init.point_number = point_number;
59.     ssl_init.sample_rate = sample_rate;
60.     ssl_init.bf_mode = 0;
61.     ssl_init.channel = MIC_NUM;
62.     ssl_config.temperature = temperature; //c
63.     ssl_config.noise_gate_dbfs = -80;
64.     ssl_config.direction_frame_num = frame_number;
65.     /******init algorithm********/
66.     handle = IaaSsl_Init((char*)WorkingBuffer2, &ssl_init);
67.     if (handle==NULL)
68.     {
69.         printf("SSL init error\n\r");
70.         return -1;
71.     }
72.     else
73.     {
74.         printf("SSL init succeed\n\r");
75.     }
76.
77.     ret = IaaSsl_Config(handle ,&(ssl_config));
78.     if (ret)
79.     {
80.         printf("Error occured in SSL Config\n\r");
81.         return -1;
82.     }
```

```
83.     ret = IaaSsl_Set_Shape(handle,shape);
84.     if (ret)
85.     {
86.         printf("Error occured in Array shape\n\r");
87.         return -1;
88.     }
89.
90.     ret = IaaSsl_Cal_Params(handle);
91.     if (ret)
92.     {
93.         printf("Error occured in Array matrix calculation\n\r");
94.         return -1;
95.     }
96.
97.     sprintf(infileName,"%s","./../sample/data/Chn_Left_right_12_0.wav");
98.     sprintf(outfileName,"%s","./SSL_result.txt");
99.
100.        fin = fopen(infileName, "rb");
101.        if(!fin)
102.        {       printf("the input file 0 could not be open\n\r");
103.            return -1;
104.        }
105.
106.        fout = fopen(outfileName, "w");
107.        if(!fout)
108.        {
109.            printf("the output file could not be open\n\r");
110.            return -1;
111.        }
112.
113.        fread(input, sizeof(char), 44, fin); // read header 44 bytes
114.        fprintf(fout,"%s\t%s\t%s\n\r","time","direction","case");
115.        while(fread(input, sizeof(short), ssl_init.point_number*2, fin))
116.        {
117.            counter++;
118.            T0  = (long)_OsCounterGetMs();
119.            ret = IaaSsl_Run(handle,input,delay_sample);
120.            if(ret != 0)
121.            {
122.                printf("The Run fail\n");
123.                return -1;
124.            }
125.            // low resolution
126.    //      if (ssl_init.bf_mode == 1)
127.    //      {
128.    //          printf("delay_sample: %d\n",delay_sample[0]);
129.    //      }
130.            T1  = (long)_OsCounterGetMs();
131.            avg += (T1-T0);
132.            if(counter == ssl_config.direction_frame_num && ssl_init.bf_mode == 0)
133.            {
134.                counter2++;
135.                counter= 0;
136.                T2  = (long)_OsCounterGetMs();
```

```
137.            ret = IaaSsl_Get_Direction(handle, &direction);
138.            T3  = (long)_OsCounterGetMs();
139.            avg2 += (T3-T2);
140.            if(ret != 0 && ret!=ALGO_SSL_RET_RESULT_UNRELIABLE && ret!
    =ALGO_SSL_RET_BELOW_NOISE_GATE&&ret!=ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
141.            {
142.                printf("The Get_Direction fail\n");
143.                return -1;
144.            }
145.            // write txt file
146.            fprintf(fout,"%f\t%d",(float)(counter2*ssl_config.direction_frame_num*0.008),direction);
147.            if (ret==0)
148.            {
149.                fprintf(fout,"\t%s\n\r","current time is reliable!");
150.            }
151.            else if (ret==ALGO_SSL_RET_BELOW_NOISE_GATE)
152.            {
153.                fprintf(fout,"\t%s\n\r","current time volume is too small!");
154.            }
155.            else if(ret==ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
156.            {
157.                fprintf(fout,"\t%s\n\r","current time delay_sample is out of range!");
158.            }
159.            else
160.            {
161.                fprintf(fout,"\t%s\n\r","current time is not reliable!");
162.            }
163.            // reset voting
164.            ret = IaaSsl_Reset_Mapping(handle);
165.            if(ret != 0)
166.            {
167.                printf("The ResetVoting fail\n");
168.                return -1;
169.            }
170.        }
171.    }
172.    avg  = avg / (float)(ssl_config.direction_frame_num*counter2);
173.    avg2 = avg2 / (float)(counter2);
174.    printf("AVG for IaaSSL_RUN is %.3f ms\n",avg);
175.    printf("AVG for IaaSSL_GetDirection is %.3f ms\n",avg2);
176.    IaaSsl_Free(handle);
177.    fclose(fin);
178.    fclose(fout);
179.    free(WorkingBuffer2);
180.    printf("Done\n");
181.    return 0;
182. }
```

- Example II: Multichannel Microphone array (Microphone input are 4x mono-channel audio)

```
183. #include <stdio.h>
184. #include <unistd.h>
185. #include <fcntl.h>
```

```
#include <string.h>
#include <sys/time.h>
#include <sys/ioctl.h>
#include <stdlib.h>
#include "AudioSslProcess.h"

#define MIC_NUM (4)
#define USE_MALLOC   (1)
typedef unsigned char           uint8;
typedef unsigned short          uint16;
typedef unsigned long           uint32;

unsigned int _OsCounterGetMs(void)
{
    struct  timeval t1;
    gettimeofday(&t1,NULL);
    unsigned int T = ( (1000000 * t1.tv_sec)+ t1.tv_usec ) / 1000;
    return T;
}

int main(int argc, char *argv[])
{
    /*********Input file init*******/
    short input[512];
    short input_tmp1[128],input_tmp2[128],input_tmp3[128],input_tmp4[128];
    char infileName[MIC_NUM][512];
    char outfileName[512];
    FILE * fin0,* fin1,* fin2,* fin3;
    FILE * fout;
    int k;
    ALGO_SSL_RET ret;
    int counter2 = 0;
    unsigned int T0,T1,T2,T3;
    float avg = 0.0;
    float avg2 = 0.0;
    /********common setting  SSL ********/
    int point_number = 128;
    float microphone_distance = 4.0;
    int temperature = 20;
    int sample_rate = 16000;
    int delay_sample[MIC_NUM-1] = {0,0,0}; //channel-1
    int shape  = 0;
    int direction = 0;
    int frame_number = 32;
    /********SSL data init********/
    int counter = 0;
#if USE_MALLOC
    char *WorkingBuffer_SSL;
    WorkingBuffer_SSL = (char*)malloc(IaaSsl_GetBufferSize());
#endif
    AudioSslInit ssl_init;
    AudioSslConfig ssl_config;
    SSL_HANDLE ssl_handle;

```

```
240.    ssl_init.mic_distance = microphone_distance;
241.    ssl_init.point_number = point_number;
242.    ssl_init.sample_rate = sample_rate;
243.    ssl_init.bf_mode = 0;
244.    ssl_init.channel  = MIC_NUM;
245.    ssl_config.temperature = temperature;
246.    ssl_config.noise_gate_dbfs = -80;
247.    ssl_config.direction_frame_num = frame_number;
248.
249.    /*******init algorithm *****/
250.    ssl_handle = IaaSsl_Init((char*)WorkingBuffer_SSL, &ssl_init);
251.    if (ssl_handle == NULL)
252.    {
253.        printf("Init fail\n\r");
254.        return -1;
255.    }
256.    else
257.    {
258.        printf("SSL init succeed\n\r");
259.    }
260.
261.    ret = IaaSsl_Config(ssl_handle ,&(ssl_config));
262.    if (ret)
263.    {
264.        printf("Error occured in SSL Config\n\r");
265.        return -1;
266.    }
267.
268.    ret = IaaSsl_Set_Shape(ssl_handle,shape);
269.    if (ret)
270.    {
271.        printf("Error occured in Array shape\n\r");
272.        return -1;
273.    }
274.
275.    ret = IaaSsl_Cal_Params(ssl_handle);
276.    if (ret)
277.    {
278.        printf("Error occured in Array matrix calculation\n\r");
279.        return -1;
280.    }
281.
282.    /********open input file and input file*****/
283.
284.    sprintf(infileName[0],"%s","./../sample/data/Chn-01.wav");
285.    sprintf(infileName[1],"%s","./../sample/data/Chn-02.wav");
286.    sprintf(infileName[2],"%s","./../sample/data/Chn-03.wav");
287.    sprintf(infileName[3],"%s","./../sample/data/Chn-04.wav");
288.    sprintf(outfileName,"%s","./SSL_result.txt");
289.    fin0 = fopen(infileName[0], "rb");
290.    if(!fin0)
291.    {
292.        printf("the input file0 could not be open\n\r");
293.        return -1;
```

```
294.    }
295.    fin1 = fopen(infileName[1], "rb");
296.    if(!fin1)
297.    {
298.        printf("the input file 1 could not be open\n\r");
299.        return -1;
300.    }
301.    fin2 = fopen(infileName[2], "rb");
302.    if(!fin2)
303.    {
304.        printf("the input file 2 could not be open\n\r");
305.        return -1;
306.    }
307.    fin3 = fopen(infileName[3], "rb");
308.    if(!fin3)
309.    {
310.        printf("the input file 3 could not be open\n\r");
311.        return -1;
312.    }
313.    fout = fopen(outfileName, "w");
314.    if(!fout)
315.    {
316.        printf("the output file could not be open\n\r");
317.        return -1;
318.    }
319.
320.    fread(input, sizeof(char), 44, fin0); // read header 44 bytes
321.    fread(input, sizeof(char), 44, fin1); // read header 44 bytes
322.    fread(input, sizeof(char), 44, fin2); // read header 44 bytes
323.    fread(input, sizeof(char), 44, fin3); // read header 44 bytes
324.
325.    short * input_ptr;
326.    fprintf(fout,"%s\t%s\t%s\n\r","time","direction","case");
327.    while(fread(input_tmp1, sizeof(short), point_number, fin0))
328.    {
329.        fread(input_tmp2, sizeof(short), point_number, fin1);
330.        fread(input_tmp3, sizeof(short), point_number, fin2);
331.        fread(input_tmp4, sizeof(short), point_number, fin3);
332.        input_ptr = input;
333.        for(k=0;k<point_number;k++)
334.        {
335.            *input_ptr =  input_tmp1[k];
336.            input_ptr++;
337.            *input_ptr =  input_tmp2[k];
338.            input_ptr++;
339.            *input_ptr =  input_tmp3[k];
340.            input_ptr++;
341.            *input_ptr =  input_tmp4[k];
342.            input_ptr++;
343.        }
344.        counter++;
345.        T0  = (long)_OsCounterGetMs();
346.        ret = IaaSsl_Run(ssl_handle,input,delay_sample);
347.        if(ret != 0)
```

```
348.         {
349.             printf("The Run fail\n");
350.             return -1;
351.         }
352.         // low resolution
353.  //     if (ssl_init.bf_mode == 1)
354.  //     {
355.  //         printf("delay_sample: %d,%d,%d\n",delay_sample[0],delay_sample[1],delay_sample[2]);
356.  //     }
357.         T1  = (long)_OsCounterGetMs();
358.         avg += (T1-T0);
359.
360.         if(counter == ssl_config.direction_frame_num && ssl_init.bf_mode == 0)
361.         {
362.             counter2++;
363.             counter= 0;
364.             T2  = (long)_OsCounterGetMs();
365.             ret = IaaSsl_Get_Direction(ssl_handle, &direction);
366.             T3  = (long)_OsCounterGetMs();
367.             avg2 += (T3-T2);
368.             if(ret != 0 && ret!=ALGO_SSL_RET_RESULT_UNRELIABLE && ret!
    =ALGO_SSL_RET_BELOW_NOISE_GATE&&ret!=ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
369.             {
370.                 printf("The Get_Direction fail\n");
371.                 return -1;
372.             }
373.             // write txt file
374.             fprintf(fout,"%f\t%d",(float)(counter2*ssl_config.direction_frame_num*0.008),direction);
375.             if (ret==0)
376.             {
377.                 fprintf(fout,"\t%s\n\r","current time is reliable!");
378.             }
379.             else if (ret==ALGO_SSL_RET_BELOW_NOISE_GATE)
380.             {
381.                 fprintf(fout,"\t%s\n\r","current time volume is too small!");
382.             }
383.             else if(ret==ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE)
384.             {
385.                 fprintf(fout,"\t%s\n\r","current time delay_sample is out of range!");
386.             }
387.             else
388.             {
389.                 fprintf(fout,"\t%s\n\r","current time is not reliable!");
390.             }
391.             // reset voting
392.             ret = IaaSsl_Reset_Mapping(ssl_handle);
393.             if(ret != 0)
394.             {
395.                 printf("The ResetVoting fail\n");
396.                 return -1;
397.             }
398.         }
399.     }
400.     avg  = avg / (float)(ssl_config.direction_frame_num*counter2);
```

```
401.      avg2 = avg2 / (float)(counter2);
402.      printf("AVG for IaaSSL_RUN is %.3f ms\n",avg);
403.      printf("AVG for IaaSSL_GetDirection is %.3f ms\n",avg2);
404.      IaaSsl_Free(ssl_handle);
405.      fclose(fin0);
406.      fclose(fin1);
407.      fclose(fin2);
408.      fclose(fin3);
409.      fclose(fout);
410.      free(WorkingBuffer_SSL);
411.      printf("Done\n");
412.
413.      return 0;
414.  }
```

3.9. IaaSsl_Get_Direction

- Features

  Get the result of Ssl algorithm processing.

- Syntax

  ALGO_SSL_RET IaaSsl_Get_Direction(SSL_HANDLE handle, int* direction);

- Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input
direction	For ULA, the value is between -90~90. For UCA, the value is between -90~270. When the value is -10000, there are three possibilities. The first is that the volume is lower than noise_gate_dbfs,and the second is that the amount of data is not enough to estimate a reliable direction, and the third is that the estimation is out of range.	Output

- Return value

Return value	Result
0	Successful
0x10000107	Successful.Warning: The estimation is out of range.
0x10000113	Successful.Warning: The volume is lower than noise_gate_dbfs.
0x10000114	Successful.Warning: The amount of data is not enough to estimate a reliable direction.

others	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

IaaSsl_Reset_Mapping must be called after using IaaSsl_Get_Direction

➢ Example

Please refer to IaaSsl_Run example.

3.10. IaaSsl_Reset_Mapping

➢ Features

Reinitialize the buffer after IaaSsl_Get_Direction.

➢ Syntax

ALGO_SSL_RET IaaSsl_Reset_Mapping(SSL_HANDLE handle);

➢ Parameters

Parameter Name	Description	Input/Output
handle	Ssl algorithm handle	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

IaaSsl_Reset_Mapping must be called after using IaaSsl_Get_Direction.

➢ Example

Please refer to IaaSsl_Run example.

3.11. IaaSsl_Reset

➢ Features

Reinitialize Ssl algorithm.

➢ Syntax

SSL_HANDLE IaaSsl_Reset(SSL_HANDLE working_buffer,AudioSslInit* ssl_init);

➢ Parameters

Parameter Name	Description	Input/Output
working_buffer	Memory address for Ssl algorithm running	Input
ssl_init	Ssl algorithm initialization structure pointer	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

➢ Example

Please refer to IaaSsl_Run example.

3.12. IaaSsl_Free

➢ Features

Release Ssl algorithm resources.

➢ Syntax

ALGO_SSL_RET IaaSsl_Free(SSL_HANDLE handle);

➢ Parameters

Parameter Name	Description	Input/Output
handle	Src algorithm handle	Input

➢ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➢ Dependency

- Header: AudioSslProcess.h
- Library: libSSL_2MIC_LINUX.so/ libSSL_2MIC_LINUX.a/libSSL_4MIC_LINUX.so/libSSL_4MIC_LINUX.a

※ Note

None.

➢ Example

Please refer to IaaSsl_Run example.

4. SSL Data Type

4.1. SSL data type list

Data type	**Definition**
AudioSslInit	Ssl algorithm initialization parameter structure type
AudioSslConfig	Ssl algorithm configuration parameter structure type
SSL_HANDLE	Ssl algorithm handle type

4.2. AudioSslInit

➢ Description

Define Ssl algorithm initialization parameter structure type.

➢ Definition

```
typedef struct
{
    unsigned int point_number;
    unsigned int sample_rate;
    float mic_distance;
    unsigned int bf_mode;
    int channel;
}AudioSslInit;
```

➢ Member

Member name	Description
point_number	The sampling points that Ssl algorithm processed once
sample_rate	Sampling rate, currently supports 8k/16k/32k/48k
mic_distance	The distance between two mics, unit: cm
bf_mode	Whether it is beamforming mode. If want to use direction from SSL, please set as 0 and take the direction from IaaSsl_Get_Direction. If want to use the delay sample from SSL, please set as 1.
channel	The number of microphones

※ Note

- If delay_sample is required, it is recommended to enable bf_mode
- If bf_mode is set as 1, IaaSsl_Get_Direction and IaaSsl_ResetVoting can't be used.
- If microphone geometry belongs to uniform linear array, the mic_distance needs to be set as the distance between adjacent microphones. If microphone geometry belongs to uniform circular array, the mic distance needs to be set as the diameter of the circle.

➢ Related data types and interfaces

IaaSsl_Init

IaaSsl_Reset

4.3. AudioSslConfig

➢ Description

Define Ssl algorithm configuration parameter structure type.

➢ Definition

```
typedef struct
{
    unsigned int temperature;
    int noise_gate_dbfs;
    int direction_frame_num;
}AudioSslConfig;
```

➢ Member

Member name	Description
temperature	Ambient temperature (Celsius) Celsius = (5/9) * (Fahrenheit-32) Step size is 1
noise_gate_dbfs	Noise gain threshold (dBfs) Note: Below this value, the frame will be treated as noise and will not enter the calculation of the direction. Step size is 1
direction_frame_num	The number of frames detected by the SSL function. Step size is 32. Note: The number of frames for SSL detection must be a multiple of 32. One frame of data processed by SSL is 128 sampling points. Time of detection once = s32DirectionFrameNum * 128 / sampling rate. For example: ： The current sampling rate is 16K, setting s32DirectionFrameNum to 32, detection time =32 * 128 / 16000=0.256(s)

※ Note

- None.

➢ Related data types and interfaces

IaaSsl_Config

IaaSsl_Get_Config

4.4. SSL_HANDLE

➢ Description

Define Ssl algorithm handle type.

➢ Definition

typedef void* SSL_HANDLE;

➢ Member

Member name	Description

※ Note

None.

➢ Related data types and interfaces

IaaSsl_Init

IaaSsl_Config

IaaSsl_Get_Config

IaaSsl_Set_Shape

IaaSsl_Cal_Params

IaaSsl_Run

IaaSsl_Get_Direction

IaaSsl_Reset_Mapping

IaaSsl_Reset

IaaSsl_Free

5. Error Code

SSL API error codes are shown as follow:

Table 5-1 SSL API error code

Error code	Definition	Description
0x00000000	ALGO_ SSL _RET_SUCCESS	SSL runs successfully
0x10000101	ALGO_ SSL _RET_INIT_ERROR	SSL initialization error

Error code	Definition	Description
0x10000102	ALGO_ SSL _RET_INVALID_CONFIG	SSL Config is invalid
0x10000103	ALGO_ SSL _RET_INVALID_HANDLE	SSL Handle is invalid
0x10000104	ALGO_ SSL _RET_INVALID_SAMPLERATE	SSL sample rate is invalid
0x10000105	ALGO_ SSL _RET_INVALID_POINTNUMBER	SSL sampling point is invalid
0x10000106	ALGO_SSL_RET_INVALID_BFMODE	bf_mode setting of SSL init is invalid
0x10000107	ALGO_SSL_RET_DELAY_SAMPLE_TOO_LARGE	Warning:The delayed sample is too large, please check the set distance and sampling rate
0x10000108	ALGO_ SSL _RET_INVALID_CALLING	SSL API call sequence error
0x10000109	ALGO_ SSL _RET_API_CONFLICT	Other APIs are running
0x10000110	ALGO_SSL_RET_INVALID_CHANNEL	SSL channel number is invalid
0x10000111	ALGO_SSL_RET_INVALID_GEOMETRY_TYPE	SSL array shape is invalid.
0x10000112	ALGO_SSL_RET_INVALID_ARRAY_TYPE	The shape of dual microphone must be 0
0x10000113	ALGO_SSL_RET_BELOW_NOISE_GATE	Warning: The volume is lower than noise_gate_dbfs
0x10000114	ALGO_SSL_RET_RESULT_UNRELIABLE	Warning: The amount of data is not enough to estimate a reliable direction.